

Transaction Management

Managing a transaction that involves multiple microservices is not easy. Distributed transaction implementation requires complex mechanism to maintain the **Atomicity, Consistency, Isolation, and Durability (ACID)** of transaction data. This is because, as per the single repository principle, each microservice has its own database, which cannot be accessed by other microservices directly. Other microservices can only access the data via APIs. There is no easy way to maintain ACID principals in multiple databases.

A very popular way of implementing distributed transactions is a two-phase commit in distributed monolithic applications. However, it has limitations. In this chapter, we'll discuss the popular approaches and patterns of implementing distributed transactions:

- Two-phase commit (2PC)
- Distributed sagas or compensating transaction
 - Feral Concurrency Control
 - Routing slips
- Distributed saga implementation